IDS Placement in RPL Networks: Project Results, Code Audit, and a Synthetic DR-FedID Prototype

Mohammadilyas Patel, Shaik Sifat Ahmed, and Abhinay
CS23B1078, CS23B1035, CS23I1009
Department of Computer Science and Engineering
IIITDM Kancheepuram, India

***Abstract*—The location of an intrusion detection system (IDS) affects what it can observe in a low-power RPL network. This report examines four placement and learning approaches presented in a course project: root-only monitoring (CIDwL), centralized collection from distributed monitors (CIDwG), distributed alarm voting (DCID), and federated intrusion detection (FedID). We compare the team's repository with its report claims and with the cited placement study. The evaluation and plotting scripts contain numerical arrays rather than a runnable Cooja experiment or a FedID training pipeline. Their stored values show a decline in root-only Blackhole accuracy from 0.988 at Level 1 to 0.515 at Level 8, while nine CIDwG monitors are assigned 0.994 accuracy and 378,000 bytes per round. The FedID table lists 0.957 accuracy at a 10% sharing ratio and zero testing bytes. Separately, the DR-FedID program generates synthetic data and produces seeded outputs, including 67.96% accuracy for its routine labeled Multi-Krum and a mean detection rate of 86.15% under its monitor-rotation heuristic. These are prototype results, not network measurements. We document the evidence behind each number, identify conflicts between scripts and figures, and outline what is needed for a full reproduction.**

***Index Terms*—RPL, intrusion detection, IDS placement, federated learning, reproducibility, synthetic evaluation.**

I. Introduction

Internet of Things deployments often rely on small battery-powered devices that exchange short messages over lossy links. Routing in these networks can be disrupted by compromised devices that still hold valid network credentials. A detector at the border router may observe traffic arriving at the root, but it cannot directly inspect a packet that an intermediate node discarded. The placement of an IDS is therefore part of the security design, alongside the classifier and the features it receives.

The Routing Protocol for Low-Power and Lossy Networks (RPL) builds a destination-oriented directed acyclic graph (DODAG) rooted at a border router. Nodes select parents and advertise a rank that reflects their position in the graph [1]. Attacks can target this control plane or the data plane. A malicious mote can advertise an artificially attractive rank, trigger a version change, selectively drop traffic, or flood control messages. A taxonomy of RPL attacks describes these and related threats [2]. Their visibility depends on where observations are collected: a root-only sensor, a network of telemetry monitors, and a set of local detectors do not receive the same evidence.

Yılmaz, Şen, and Aydoğan studied IDS placement in RPL and compared centralized, distributed, and federated approaches [3]. The Team 19 project adopts similar research questions and includes code, figures, a parameter file, a presentation, and a paper. The repository's README describes Cooja experiments and a DR-FedID extension. However, the files do not all provide the same level of evidence: some scripts print arrays, some figures are built from constants, and the DR-FedID program generates its own synthetic samples.

This report has two aims. First, it explains the placement trade-offs represented by the project's numbers and figures. Second, it traces those results to the files that produce them, so that a reader can tell a published result from a stored summary or a synthetic prototype output. We address five questions:

- How do the stored CIDwL values change as an attacker is placed farther from the root?
- What accuracy and nominal byte totals does the CIDwG monitor-count table report?
- How do the DCID threshold arrays trade detection accuracy against false-positive rate?
- What relationship do the FedID summary arrays show between data sharing, cosine similarity, and Blackhole accuracy?
- What does the DR-FedID script actually simulate, and how far do its outputs support the claims made for it?

The aim is not to reproduce the cited paper's experiments. We inspect repository commit `66647fb37a563b63da6d8707b4ec09f9607f803d`, run the provided summary and synthetic programs where possible, regenerate the plots in an isolated folder, and compare those outputs with the submitted figures. We do not have the per-run Cooja traces or the CICIoT2023 data split used for the reported benchmark values. Consequently, conclusions about those measurements remain limited to what the source study and the checked-in files state.

II. Background and Related Work

A. RPL operation and attack visibility

RPL organizes routes as a DODAG. The root is typically connected to an IPv6 network through a 6LoWPAN border router, while forwarding nodes send child traffic upward toward that root. Each node maintains a preferred parent and a rank; the exact route selection depends on the objective function. The repository configuration names MRHOF and

Contiki 2.7, but those entries describe intended settings and are not proof that a particular simulator run used them.

The project's configuration lists seven attack labels. Decreased Rank (DR) attempts to attract descendants by advertising an artificially low rank. Increased Version (IV) changes the DODAG version, potentially causing repair activity. A Blackhole (BH) node drops forwarded packets, while Selective Forwarding (SF) drops only part of the traffic. DIS and DIO flooding target control-message handling. The Wormhole/P2P Bypass (WP) label covers a tunnel that makes distant nodes appear connected by a shorter path. These names describe attack scenarios, but the repository does not include packet traces or attack implementations that would let us inspect their timing and intensity.

Earlier IDS work addresses related problems through different observation and classification choices. Verma and Ranga examine ensemble learning for RPL intrusion detection [4]; Napiah et al. describe a 6LoWPAN header analyzer [5]; and Shukla studies a machine-learning approach to wormhole detection [6]. SVELTE is an example of a real-time IDS designed for constrained IoT networks [7]. Other cited work explores trust-based distributed detection [8], [9]. These systems motivate the placement question, but their results should not be treated as direct baselines unless they share the same topology, attack definitions, features, and evaluation split.

B. Federated learning and heterogeneous data

Federated averaging combines client updates while keeping each client's raw training examples local [10]. This can reduce central collection of features, although model updates still require communication during training. A zero-byte testing figure, therefore, says nothing about the cost of training rounds or model distribution. Nor does federated learning by itself prove a formal privacy guarantee.

The project frames depth-dependent traffic differences as non-identically distributed (non-IID) data. This is plausible in a multi-hop graph: a node near the root may forward transit traffic for many descendants, while a leaf sees mostly its own flows. Federated optimization is known to be affected by heterogeneous client data [11]. The project stores cosine-similarity and accuracy values against a sharing ratio, but does not include the client datasets, model updates, or an executable FedID pipeline. The arrays can illustrate a claimed trend; they cannot establish how the trend was measured.

The repository describes local XGBoost models and Flower-based aggregation. XGBoost is a gradient-boosted tree system [12], and Flower provides tooling for federated-learning experiments [13]. Neither package is invoked by the checked-in architecture evaluator. Thus the implementation audit below separates the method described in the paper from the code that is actually available.

III. Network Model and IDS Architectures

A. Topology represented in the project

The configuration specifies 30 nodes arranged in a 10-by-3 grid and 10 DODAG depth levels. The topology drawing numbers nodes from 0 to 29 and shows parent arrows toward a root. A separate illustration highlights example monitor and attacker placements. These drawings communicate the intended setup; they are not snapshots from Cooja and do not provide link-quality or packet-loss measurements. Cooja and Contiki are appropriate tools for sensor-network simulation, but a runnable project and its configuration are needed to verify an experiment [14], [15].

TABLE I
Settings declared in the repository configuration.

Area	Declared setting
Network	30 nodes; 10 depth levels; 10-by-3 grid
Routing	RPL with MRHOF; IEEE 802.15.4 named
Software	Contiki 2.7 and Cooja named
Features	60-second window; 35 routing/traffic features
FedID	XGBoost; depth 6; 50 estimators; learning rate 0.1; sharing ratio 10%
DR-FedID	120-second shuffle interval; Byzantine tolerance 2; epsilon 1.5, delta 0.00001

Table I summarizes the settings declared in configs/simulation_params.yaml. The evaluation and graph scripts do not load this YAML file. We therefore refer to these entries as configuration declarations. They should not be read as a verified log of the environment used to generate the hard-coded results.

B. Placement and information flow

Four labels appear throughout the project. In CIDwL, a classifier at the root uses its own local observations. CIDwG adds monitors that send feature summaries to a central detector. In DCID, local monitors produce binary alarms and the root applies a voting rule. FedID is described as training models at multiple sites and aggregating model information rather than forwarding every feature at test time. The configurations and comparison script use these labels, but only the evaluator's fixed tables are implemented as executable summary logic.

For DCID, the root can be described as raising an alarm when the mean of the monitors' binary alarm values reaches a chosen threshold, theta. The code lists a fixed communication value of 2,700 bytes per round; it does not serialize alarms, account for headers or retransmissions, or derive that total from a packet capture.

In federated averaging, the server combines each client update using a weight proportional to that client's number of training examples. In notation, client k's update receives weight n_k/N, where N is the total number of client examples. This explains the reported learning paradigm; the repository does not implement the calculation for XGBoost. A prose method description is not a substitute for the code, data partition, and aggregation rules needed to reproduce it.

IV. Materials and Review Method

We reviewed the repository at commit 66647fb37a563b63da6d8707b4ec09f9607f803d [16]. The main files were

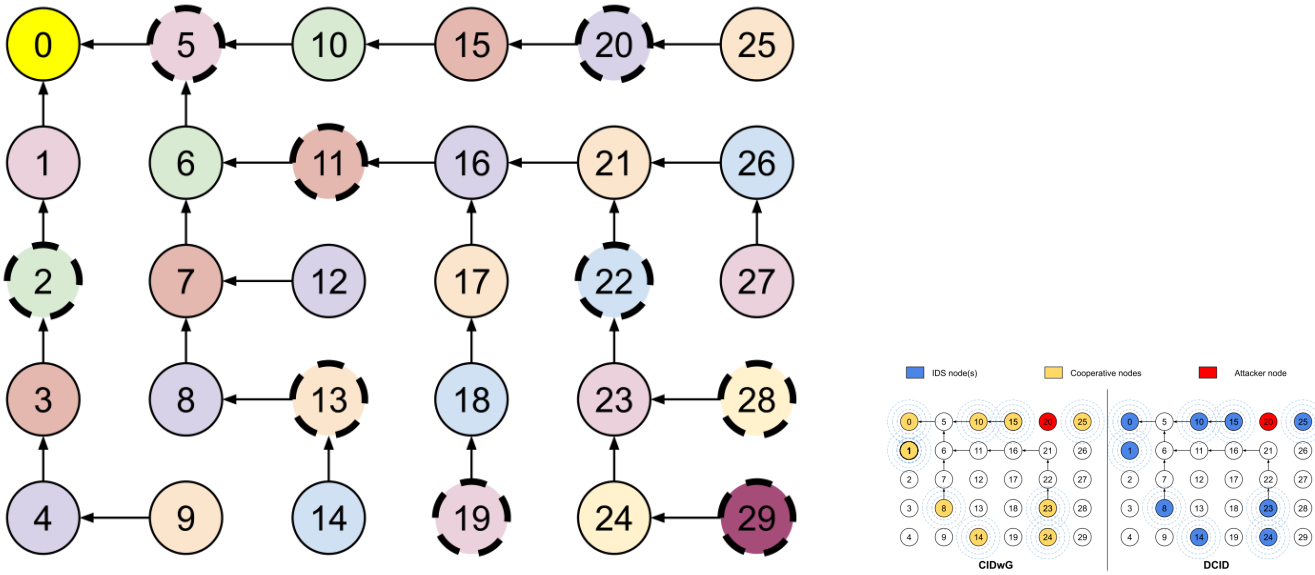


Fig. 1. Repository illustrations of the 30-node routing topology and example CIDwG/DCID placements. The diagrams describe the intended scenario; they do not constitute simulator output.

TABLE II
ARCHITECTURAL COMPARISON AS DESCRIBED IN THE PROJECT. VALUES ARE REPOSITORY SUMMARIES, EXCEPT WHERE THE METHOD IS ATTRIBUTED TO THE CITED STUDY.

Approach	Decision location	Information exchanged	Code-level cost entry	Evidence available
CIDwL	Root/6LBR	Root-local observations	0 bytes in comparison summary	Fixed depth arrays
CIDwG	Root with collaborators	Feature telemetry to root	42,000 bytes per monitor per round	Fixed accuracy and byte arrays
DCID	Local monitors and root	Binary alarms for a vote	2,700 bytes per round	Fixed threshold and FPR arrays
FedID	Distributed training; root inference described	Model information during training	0 testing bytes in summary	Fixed similarity/accuracy arrays; no trainer
DR-FedID	Adaptive placement/aggregation proposed	Synthetic updates and monitor scores in script	Not measured as a network cost	Seeded synthetic program; no LDP

src/evaluate_architectures.py, src/generate_result_graphs.py, src/simulate_dr_fedid.py, and configs/simulation_params.yaml. We also inspected the README, presentation and paper images. The similarly named files under scripts/ match the three files under src/ at this commit, but the graph images with the same filenames in figures/ and paper/images/ are not identical.

We used four evidence labels in interpreting the material:

1) *Source-study result*: a claim attributed to the cited Yılmaz et al. article [3].
2) *Stored repository value*: a literal array, table entry, or print string in the Team 19 code.
3) *Synthetic program output*: a value calculated by the DR-FedID script from its generated samples and seeded random numbers.
4) *Unverified claim*: a result or guarantee that cannot be traced to a source-study result or to a calculation in the checked-in code.

The main evaluator can be run with pythonsrc/evaluate _architectures.py; it prints the RQ tables and comparison strings. It does not call a simulator or load a data file. The graph generator plots arrays written in the source. We regenerated its figures in a clean working folder and treated them as visualizations of those arrays. The DR-FedID script is different: it creates artificial samples and computes outputs. We ran that program with its fixed random seed and report the resulting values as outputs of that synthetic setup. None of these steps reconstructs the source paper's 31,626 Cooja scenarios.

The evidence boundary is summarized in Table III. A screenshot of console output can confirm what a script printed, but it cannot establish how an array was originally measured. Likewise, a figure file can be recreated from code constants without providing the underlying run records.

TABLE III
WHAT THE AVAILABLE PROJECT FILES ESTABLISH.

Material	Supports / does not establish
Evaluator script	Supports exact printed arrays; does not show their raw measurement source
Graph script	Supports plotted values; does not calculate them from traces
Configuration YAML	Supports intended settings; scripts do not load it
DR-FedID program	Supports reproducible synthetic outputs for its seed; does not model RPL packets
Screenshots	Support that outputs/plots were displayed; do not provide per-run experiment records
Source article	Supports its own published study and benchmark claims; not an independent rerun by Team 19

V. RESULTS: CENTRALIZED AND DISTRIBUTED PLACEMENT

A. RQ1: Root-only detection across depth

The evaluator stores three nine-level accuracy series: Blackhole, Decreased Rank, and DIS Flooding. Table IV contains the values printed by the evaluator. The Blackhole series moves from 0.988 at Level 1 to 0.515 at Level 8, then ticks up to 0.520 at Level 9. The change from Level 1 to Level 8 is 47.3 percentage points, or a relative decrease of approximately 47.9%. This is not a strictly decreasing sequence because of the Level 9 increase.

The Decreased Rank values fall from 0.991 to 0.605 between the first and ninth levels. DIS Flooding falls from 0.995 to 0.660 over the same interval. These trends are consistent with the idea that a root-only detector receives less direct evidence about attacks farther down a routing tree. The code does not isolate that mechanism experimentally, so we treat observability as an interpretation of the placement pattern, not as a measured causal explanation.

The heatmap in Fig. 2 displays seven attack classes and rounds each value to two decimal places. Only three attack series appear in the evaluator's printed RQ1 table. The other four heatmap rows are separate constants in the graph generator. The heatmap therefore broadens the visual summary but does not add independently verified attack measurements. This distinction is important for the SF row: the graph shows values below 0.50 at the deepest levels, while the main evaluator does not print a corresponding SF series.

B. RQ2: CIDwG monitor count and nominal traffic

The evaluator lists a Blackhole accuracy value for one through nine monitors and computes communication as 42,000 bytes multiplied by the monitor count. The stored accuracy rises from 0.682 for one monitor to 0.994 for nine. The corresponding nominal byte entry rises from 42,000 to 378,000 bytes per round. At eight monitors, the table lists 0.989 and 336,000 bytes; at nine, it lists 0.994 and 378,000 bytes.

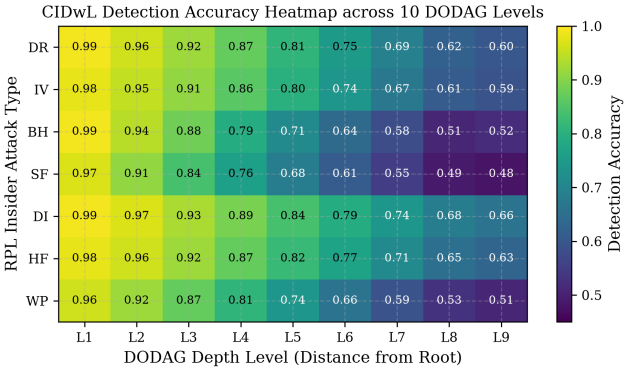


Fig. 2. Seven CIDwL attack series defined in the graph generator and rounded to two decimals.

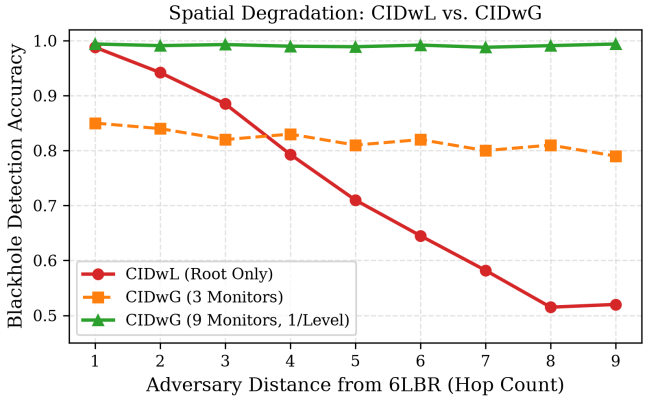


Fig. 3. Hard-coded spatial series for CIDwL, three-monitor CIDwG, and nine-monitor CIDwG.

These numbers describe the table's assumed scaling. They do not measure radio occupancy, energy, latency, packet loss, or the impact of contention. The earlier manuscript described 378,000 bytes as saturating IEEE 802.15.4 channels. The repository provides no airtime calculation, MAC framing assumptions, measured channel utilization, or loss trace to establish saturation. We therefore report the byte count as a nominal code value and leave channel impact as an open measurement question.

The separate spatial plot in Fig. 3 contains another pair of CIDwG series: three-monitor accuracy ranges from 0.85 to 0.79 across attacker distance, while the nine-monitor series remains close to 0.99. These per-level arrays are distinct from the monitor-count table. For example, the monitor-count table gives a single value of 0.824 for three monitors. The repository does not explain the aggregation relationship between these representations, so they should not be combined as if they were observations from one documented run.

C. RQ3: DCID voting threshold

The evaluator lists five thresholds (0.10, 0.30, 0.50, 0.70, and 0.90), while the plotting script includes nine thresholds and an FPR array. We combine the nine plotted points in

TABLE IV
CIDWL VALUES PRINTED BY SRC/EVALUATE_ARCHITECTURES.PY. THE REPOSITORY HAS NO RUN-LEVEL TRACES FOR RECALCULATING THESE FIGURES.

Attack	L1	L2	L3	L4	L5	L6	L7	L8	L9
Blackhole	.988	.942	.885	.793	.710	.645	.582	.515	.520
Decreased Rank	.991	.965	.920	.871	.812	.750	.690	.620	.605
DIS Flooding	.995	.970	.935	.890	.840	.790	.740	.680	.660

TABLE V
CIDWG MONITOR-COUNT VALUES AND NOMINAL BYTES PER ROUND IN THE EVALUATOR.

Monitors	BH accuracy	Bytes/round
1	.682	42,000
2	.751	84,000
3	.824	126,000
4	.889	168,000
5	.931	210,000
6	.960	252,000
7	.978	294,000
8	.989	336,000
9	.994	378,000

TABLE VI
DCID ARRAYS IN THE EVALUATOR AND GRAPH GENERATOR. FPR VALUES COME FROM THE GRAPH GENERATOR.

Threshold	Accuracy	FPR	Bytes/round
.10	.945	.185	2,700
.20	.920	.120	2,700
.30	.892	.075	2,700
.40	.850	.045	2,700
.50	.805	.021	2,700
.60	.740	.012	2,700
.70	.672	.006	2,700
.80	.590	.003	2,700
.90	.510	.001	2,700

Table VI, marking them as values stored in code rather than recomputed rates. The communication figure stays at 2,700 bytes for every threshold.

As the threshold increases, the stored detection accuracy falls and the stored FPR also falls. At theta=0.10, the graph lists 0.945 accuracy and 0.185 FPR; at theta=0.90, it lists 0.510 and 0.001. The 0.50 point is 0.805 accuracy and 0.021 FPR. This is a useful operating point to discuss, but the code supplies no cost function, target FPR, or search procedure that would make it a demonstrated optimum. It is more accurate to describe it as one selected trade-off point.

Accuracy and FPR ordinarily require a labeled evaluation set and counts of true positives, false positives, true negatives, and false negatives. None of those records is included. We can check that the arrays move in the stated directions, but we cannot recalculate either metric or determine the uncertainty around a point. Figure 4 is thus a plotted representation of the script's fixed trade-off values.

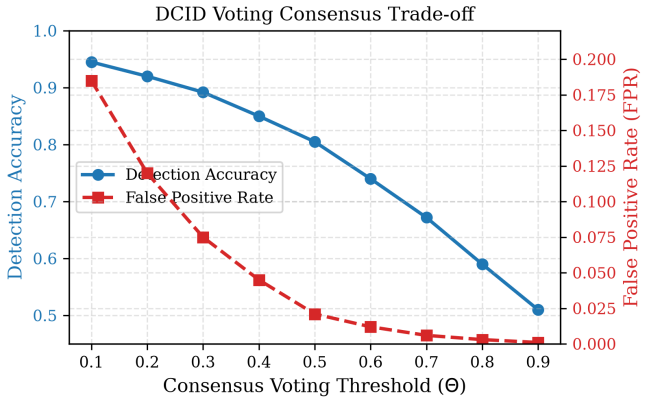


Fig. 4. Accuracy and FPR arrays plotted against the DCID voting threshold.

VI. FEDID SUMMARY AND CICIOT2023 CLAIMS

The evaluator prints six data-sharing ratios with a cosine-similarity value and a Blackhole accuracy value. In that table, cosine similarity rises from 0.54 at 0% sharing to 0.92 at 10%, while accuracy rises from 0.712 to 0.957. At 50% sharing, the listed values are 0.98 and 0.980. This pattern is compatible with the project's interpretation that more shared local data can reduce client divergence. Since the code does not define a sharing operation, provide local datasets, or train any models, it remains a pattern in the stored summary rather than a verified effect of a 10% sharing intervention.

The graph generator plots eight sharing ratios, including 15% and 40%, which do not appear in the evaluator table. At the overlapping points, the plotted values broadly follow the table; the additional points come only from the graph's constants. Its vertical marker labels 10% as "optimal," but there is no search or objective calculation in the code. We keep the figure because it conveys the project's intended trend, while treating the marker as an annotation rather than a validated optimum.

The evaluator also prints 99.60% binary accuracy and 98.80% multiclass accuracy for CICIoT2023. Those values are fixed strings in the script. They are reported in the cited placement study [3], but the Team 19 repository contains no CICIoT2023 data, split file, preprocessing pipeline, or evaluation code. We therefore attribute the benchmark figures to the cited study and do not present them as an independent project rerun. CICIoT2023 itself is a large IoT traffic collection built using 105 devices and 33 attack types [17]; reproducing

TABLE VII
FEDID VALUES PRINTED BY THE EVALUATION SCRIPT. TESTING COMMUNICATION IS A SEPARATE COLUMN FROM TRAINING COMMUNICATION.

Sharing	Cosine sim.	BH accuracy	Testing bytes
0%	.54	.712	0
5%	.78	.845	0
10%	.92	.957	0
20%	.95	.968	0
30%	.97	.975	0
50%	.98	.980	0

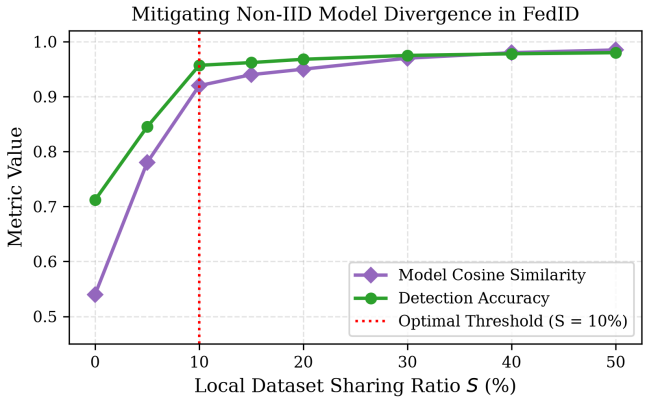


Fig. 5. FedID sharing-ratio series regenerated from the arrays in src/genera te_result_graphs.py.

a result on it also requires documenting class handling, feature selection, split strategy, and metrics.

A. Why zero testing bytes is a narrow claim

The comparison table describes FedID as sending model weights and assigns zero bytes to testing. That can be meaningful if each node can perform local inference and the global model is already present at the inference location. It does not imply zero communication over the entire lifecycle. Training may require model updates and broadcasts; model distribution, synchronization, retries, and update sizes also contribute. The evaluator has no training-byte accounting and no model file. Accordingly, our interpretation is limited to the literal label "testing bytes" in the stored output.

VII. DR-FEDID: WHAT THE PROTOTYPE COMPUTES

The repository contains a separate script that generates the results plotted in Fig. 6. We ran it with the script's seed of 42 in an isolated output folder. The script creates 10 depth-specific datasets, each with 300 synthetic benign examples and 150 synthetic attack examples. Every example has 35 generated features. A depth-dependent shift is added to the feature distribution; attack examples receive further shifts in selected features. The code divides each level's samples into a 75% training subset and a 25% test subset, then combines the local test subsets for global evaluation.

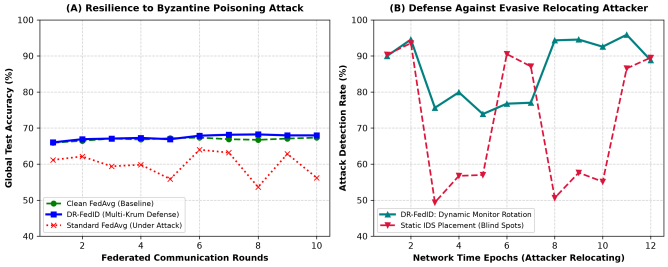


Fig. 6. Seeded synthetic outputs from src/simulate_dr_fedid.py: (a) global accuracy across ten rounds and (b) heuristic detection scores for a relocating attacker.

The aggregation experiment trains a linear sigmoid classifier with gradient descent. It runs ten communication rounds over nine collaborators. One participant is designated Byzantine and submits a deliberately perturbed vector. The output at round ten is 67.35% accuracy for clean FedAvg, 56.19% for the poisoned FedAvg condition, and 67.96% for the routine labeled DR-FedID/Multi-Krum. These values are reproducible for this synthetic setup and seed. They do not measure XGBoost behavior, a Cooja network, a physical IDS, or generalization to a real traffic distribution.

The aggregation code also deserves closer review. It sets f=1, even though the YAML declares tolerance of two Byzantine participants. For nine updates and f=1, the script's slice sorted_d[1:n-f] sums seven non-self distances; the Krum score in the cited formulation is based on the n-f-2=6 nearest other vectors [18]. The program then averages eight selected vectors. We therefore describe this as the script's routine labeled Multi-Krum, rather than claiming that the implementation satisfies the cited algorithm's resilience conditions. A formal comparison with the algorithm and tests against multiple adversarial strategies would be required before making a Byzantine-resilience claim.

The second experiment in the script runs for 12 time steps. The attacker depth follows a fixed cycle. A static configuration uses two monitor levels; the rotating configuration assigns three levels at each step. Detection scores are then drawn from ranges selected by a proximity rule. With the seed fixed at 42, the means are 72.00% for static placement and 86.15% for rotation, a 14.15 percentage-point difference (rounded to 14.2 points in the console). These are outputs of a hand-coded coverage heuristic. They are not measured from packets, and the code does not compute routing churn or entropy even though the configuration names a churn threshold.

Several other parts of the DR-FedID story are not implemented. The YAML lists epsilon and delta, but the script does not use them, clip updates, or add privacy noise. No local differential-privacy guarantee follows from the current program. The README and comparison strings also show a 98.2% DR-FedID accuracy value; that value does not match the 67.96% round-ten output of the synthetic script. It is another fixed summary that should not be presented as a result of the checked-in simulation. The architecture drawing is best

understood as a proposal for a future system.

The proposal diagram also names local XGBoost training, gradient sparsification, quantized parameter exchange, and periodic model distribution. Those mechanisms are not implemented by the current script, which trains a simple linear sigmoid model and does not account for model-transfer bytes. Figure 7 is included as a design sketch so readers can compare the intended system with the prototype that is actually present.

A. A conflict in the project's multi-criteria comparison

The graph generator's multi-criteria figure combines the CIDwL value at Level 8, the CIDwG three- and nine-monitor summaries, the DCID 0.50 point, a FedID 10% summary, and a DR-FedID value of 0.982. It places accuracy and testing communication on different axes. The script supplies no common evaluation split or cost accounting for these systems, and the DR-FedID 0.982 value conflicts with the synthetic program's round-ten output. We therefore do not use the bar heights to rank the architectures. Figure 8 is included as an example of a project visualization whose assumptions need to be made explicit before it can support a cross-system conclusion.

VIII. Discussion

The code's stored arrays tell a coherent qualitative story: root-only accuracy is lower at deeper positions for several attack types; the CIDwG table assigns greater accuracy to larger monitor sets and scales byte totals linearly; and the DCID graph trades detection against false alarms as the vote threshold changes. The FedID summary shows higher cosine similarity and accuracy at higher sharing ratios. These patterns can guide questions for a full experiment, but their current evidence is uneven. The evaluator and graph scripts define arrays; they do not supply the records behind the arrays.

There is no single placement choice that can be selected from accuracy alone. CIDwL avoids forwarding telemetry but can miss remote events. CIDwG makes more observations available to a root detector at a larger nominal communication total. DCID reduces the claimed message payload to alarm tokens, but its FPR and accuracy values need raw confusion counts. FedID shifts communication toward training and model distribution; its listed zero testing bytes do not include those stages. These are useful engineering dimensions, but a fair comparison would apply each architecture to the same topology, attack traces, train/test split, and metric definitions.

A. Validity and reproducibility limitations

Several limits affect interpretation:

- **No raw Cooja artifacts.** The repository lacks simulator projects, firmware, run seeds, traffic traces, and per-run labels. The named setup is not enough to replay a scenario.
- **No metric denominators.** Without confusion matrices or predictions, accuracy and FPR cannot be checked, and no confidence interval can be computed.
- **Different figures use different arrays.** The evaluator and graph generator have overlapping but non-identical point sets. The graph images under figures/ also differ from same-named files under paper/images/; a figure manifest would remove this ambiguity.
- **No measured resource cost.** The byte totals do not account for packet headers, radio retransmission, contention, or energy. Testing bytes are not training bytes.
- **No deployed FedID model.** The project does not include a training pipeline, client data partitions, model checkpoints, or CICIoT2023 preprocessing details.
- **Synthetic DR-FedID assumptions.** Its seed makes the program repeatable, but the generated distributions and proximity rule define the results. Repetition of the same synthetic recipe is not external validation.

The DR script sets a machine-specific Matplotlib configuration path, which complicates portability. The graph generator has the same issue. Both scripts should accept an output/cache path or use a temporary directory chosen at runtime. The configuration YAML should also be loaded by the programs rather than maintained as a separate description. These changes would reduce the chance that the paper, plots, and executable settings drift apart.

IX. A Reproduction Plan

A stronger evaluation can be built from the project structure without presuming the current arrays are measured results. The first step is to define the experiment and archive its inputs. Each scenario should record the simulator and firmware version, topology file, routing parameters, attack node and behavior, random seed, feature-window settings, and software dependencies. A machine-readable run manifest can link each trace to its configuration.

The placement experiment should then produce one labeled record per detector decision. For every attack and depth, the report should publish the confusion matrix and calculate accuracy, recall, precision, and FPR from that matrix. Repeating each scenario under multiple seeds would allow means and uncertainty intervals to be reported. The specific values in the current arrays can serve as hypotheses to check, not expected outputs to force.

For CIDwG and DCID, communication should be measured from packet captures or a documented byte model that states whether headers, retransmissions, and control traffic are included. Radio airtime and energy should be measured or estimated from an explicit MAC model before claiming channel saturation or battery impact. For FedID, the training protocol should name the client partition, update size, aggregation step, model serialization, number of rounds, and training/test communication separately. Any privacy claim should be tied to an implemented mechanism and its formal parameters.

The DR-FedID prototype should be treated as an exploratory algorithm exercise until it uses a validated aggregator and a realistic data source. A follow-up should test multiple Byzantine fractions and attack strategies, validate the selection rule against a reference implementation, and report clean,

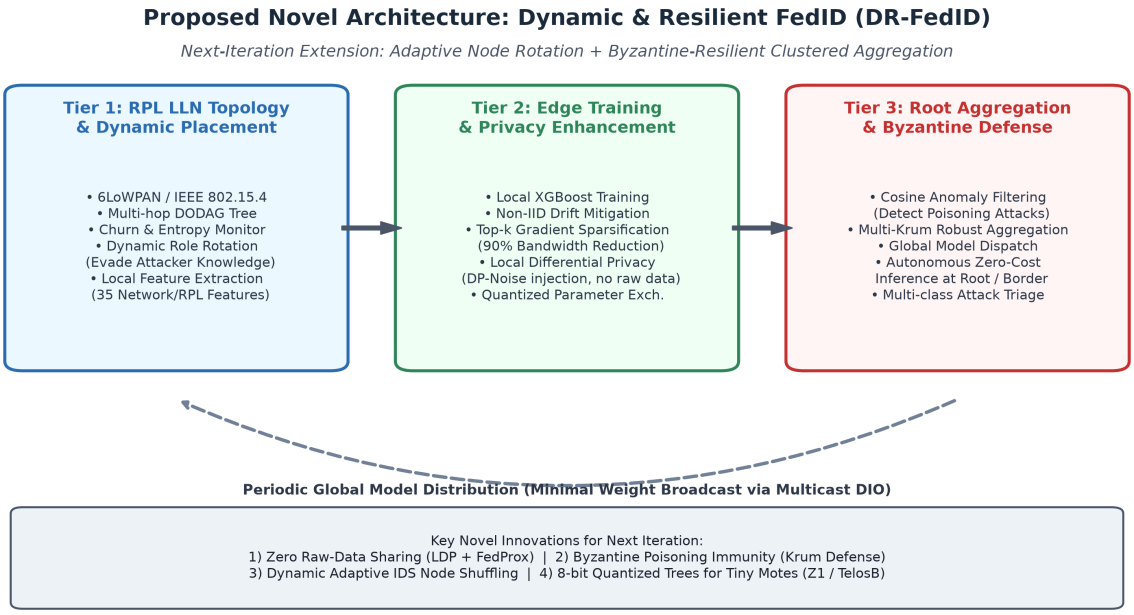


Fig. 7. DR-FedID architecture proposed in the repository. The figure describes next-iteration mechanisms; the checked-in program does not implement the complete design.

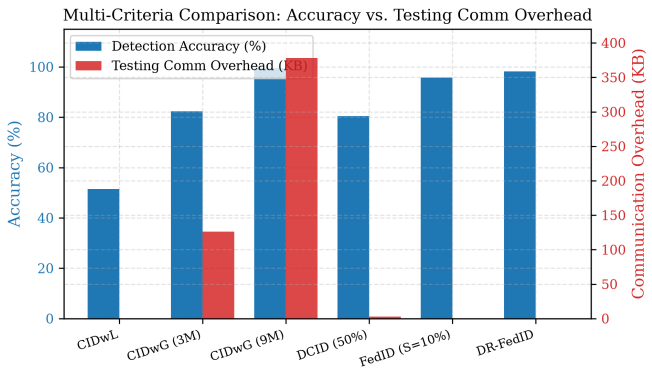


Fig. 8. Repository comparison plot. The DR-FedID bar and zero-testing-byte labels are fixed entries and are not verified by the synthetic program or a shared evaluation.

TABLE VIII
MINIMUM ADDITIONS NEEDED TO REPRODUCE THE MAIN RESULT FAMILIES.

Result family	Additional artifacts or measurements
CIDwL/CIDwG	Cooja project, firmware, seeds, raw traces, attack schedules, per-depth confusion matrices
DCID	Local predictions, alarm packets, vote rule, false-positive denominators, byte/airtime accounting
FedID	Dataset version and split, client partitions, preprocessing, trainer, model format, update sizes
DR-FedID	Validated robust aggregator, executed privacy mechanism, routing-churn inputs, repeated runs
All figures	One canonical data file per figure, script version, units, rounding rule, and generation command

poisoned, and defended runs across seeds. Monitor rotation should be triggered by a computed routing-churn statistic and evaluated against packet-level observations. These steps would turn a deterministic demonstration into an experiment that can be challenged and reproduced by another team.

X. CONCLUSION

This project raises a practical question for RPL security: what can an IDS see from its chosen location, and what does it cost to share information across constrained links? The repository's stored CIDwL, CIDwG, DCID, and FedID values illustrate several possible trade-offs. Our audit shows, however, that most of those values are fixed arrays rather than results that can be recomputed from included network records. The DR-FedID script does compute outputs, but it does so on synthetic data and a hand-coded monitor heuristic; its privacy settings are unused and its algorithm label needs validation.

The report therefore distinguishes source-study results, repository summaries, and synthetic program output. That distinction does not diminish the project's value as a course investigation; it makes the next engineering step clearer. A runnable Cooja setup, archived traces, transparent metric calculations, consistent figure inputs, and a complete FedID/DR-FedID implementation would allow the proposed comparisons to be tested rather than inferred from stored values.

REFERENCES

[1] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J. Vasseur, and R. Alexander, "RPL: IPv6 routing protocol for low-power and lossy networks," *Internet Engineering Task Force (IETF), RFC 6550*, 2012. [Online]. Available: https://www.rfc-editor.org/info/rfc6550

[2] A. Mayzaud, R. Badonnel, and I. Chrisment, "A taxonomy of attacks in RPL-based Internet of Things," *International Journal of Network Security*, vol. 18, no. 3, pp. 459–473, 2016.

[3] S. Yılmaz, S. Sen, and E. Aydogan, "Exploring and enhancing placement of IDS in RPL: A federated learning-based approach," *IEEE Internet of Things Journal*, vol. 12, no. 13, pp. 22 945–22 961, July 2025.

[4] A. Verma and V. Ranga, "ELNIDS: Ensemble learning based network intrusion detection system for RPL based Internet of Things," in *Proceedings of the 4th International Conference on Internet of Things: Smart Innovation and Usages (IoT-SIU)*, 2019, pp. 1–6.

[5] M. N. Napiah, M. Y. I. Idris, R. Ramli, and I. Ahmedy, "Compression header analyzer intrusion detection system (CHA-IDS) for 6LoWPAN communication protocol," *IEEE Access*, vol. 6, pp. 16 623–16 638, 2018.

[6] P. Shukla, "ML-IDS: A machine learning approach to detect wormhole attacks in Internet of Things," in *Proceedings of the Intelligent Systems Conference (IntelliSys)*, 2017, pp. 234–240.

[7] S. Raza, L. Wallgren, and T. Voigt, "Svelte: Real-time intrusion detection in the Internet of Things," *Ad Hoc Networks*, vol. 11, no. 8, pp. 2661–2674, 2013.

[8] Z. A. Khan and P. Herrmann, "A trust based distributed intrusion detection mechanism for Internet of Things," in *Proceedings of the IEEE 31st International Conference on Advanced Information Networking and Applications (AINA)*, 2017, pp. 1169–1176.

[9] F. Medjek, D. Tandjaoui, I. Romdhani, and N. Djedjig, "A trust-based intrusion detection system for mobile RPL based networks," in *Proceedings of the IEEE International Conference on Internet of Things (iThings)*, 2017, pp. 735–742.

[10] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017, pp. 1273–1282.

[11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, pp. 429–450, 2020.

[12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[13] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. H. Li, T. Parcollet, P. P. B. de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," *arXiv preprint arXiv:2007.14390*, 2020.

[14] F. Osterlind, A. Dunkels, J. Eriksson, N. Finne, and T. Voigt, "Cross-level sensor network simulation with Cooja," in *Proceedings of the 31st IEEE Conference on Local Computer Networks (LCN)*, 2006, pp. 641–648.

[15] A. Dunkels, B. Gronvall, and T. Voigt, "Contiki - a lightweight and flexible operating system for tiny networked sensors," in *Proceedings of the 29th Annual IEEE International Conference on Local Computer Networks (LCN)*, 2004, pp. 455–462.

[16] M. Patel, S. S. Ahmed, and Abhinay, "Cns project team 19 repository," https://github.com/mdilyas-27/cns_project_team_19/tree/66647fb37a563b63da6d8707b4ec09f9607f803d, 2026, reviewed at commit 66647fb37a563b63da6d8707b4ec09f9607f803d.

[17] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, p. 5941, 2023.

[18] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 119–129.